

Reproducing FUR: A Closer Look at Parametric Faithfulness

Group 1 – Final Report

Jialong Chen Tianle Chen Junyan Liu Kengyi Wang Wanyi Zhou

School of Data Science, Fudan University

Abstract

Faithfulness by Unlearning Reasoning steps (FUR) measures whether a language model’s chain-of-thought reasoning is faithful to its parametric knowledge by erasing reasoning-step content from model parameters and observing the effect on predictions. We reproduce FUR on a 2×2 model–dataset grid and confirm that both the absolute faithfulness scores and the cross-cell trends reported by Tutek et al. (2025) transfer to an independent implementation. We then probe the FUR loss function along two axes: swapping the forget objective and scaling the retain weight λ . Replacing the default NPO loss with SimNPO yields higher unlearning efficacy at comparable specificity while removing the need for a reference model. A logit-flattening alternative (LMF) does not reliably transfer, revealing a mismatch between local next-token uncertainty and the sequence-level forgetting signal FUR requires. Sweeping λ shows that the measured faithfulness is largely invariant across a wide range of retain weights; λ primarily governs the trade-off between unlearning strength and preservation of unrelated behavior, meaning practitioners need not fine-tune this hyperparameter to obtain a stable faithfulness measurement.

1 Introduction

Chain-of-thought (CoT) prompting elicits step-by-step reasoning from language models and often improves performance on complex tasks (Kojima et al., 2022; Wei et al., 2022). A natural follow-up question is whether the verbalized reasoning is *faithful* to the model’s actual decision process, or merely a plausible-sounding post-hoc rationalization (Jacovi and Goldberg, 2020; Turpin et al., 2023).

Most prior work probes faithfulness by perturbing the CoT in the input context—truncating steps, inserting mistakes, or paraphrasing—and checking whether the prediction changes (Lanham et al.,

2023; Bentham et al., 2024; Madsen et al., 2024). Because these methods keep the model parameters fixed, a model can potentially recover the corrupted information from its own weights, making it difficult to distinguish genuine reasoning dependence from mere context sensitivity (Parcalabescu and Frank, 2024; Neeman et al., 2023).

Tutek et al. (2025) address this confound by introducing *Faithfulness by Unlearning Reasoning steps* (FUR). Instead of perturbing the input, FUR uses machine unlearning to erase the information associated with a reasoning step from the model parameters and then measures whether the prediction changes. This *parametric faithfulness* signal is stronger than contextual perturbation: if the model cannot retrieve the erased knowledge from either context or parameters, a prediction change is direct evidence that the step was internally used.

In this work, we reproduce FUR on a 2×2 model–dataset grid (Llama-3.2-3B-Instruct and Phi-3-mini-4k-Instruct on OpenBookQA and StrategyQA) and probe its loss-function design. The FUR objective combines a forget term (NPO) with a KL retain regularizer (Figure 1). We treat this as one point in a design space and vary both components: swapping the forget objective to SimNPO (Fan et al., 2024) and LMF (Entesari et al., 2025), and sweeping the retain weight λ . Our main findings are:

1. Our reproduction closely matches the original FF-HARD values and preserves the cross-cell ranking across both datasets and models.
2. Swapping the forget objective to SimNPO yields higher unlearning efficacy without a reference model, while LMF does not reliably transfer due to the mismatch.
3. FF-HARD is largely insensitive to the retain weight λ , with λ primarily affecting the efficacy–specificity trade-off rather than the measured faithfulness.

2 Background and Related Work

This section situates FUR within prior work on chain-of-thought faithfulness. We first review why contextual perturbation methods may fail to reflect a model’s internal decision process, then distinguish them from parametric interventions.

CoT faithfulness. Despite the empirical success of CoT prompting (Wei et al., 2022; Sprague et al., 2025), it remains unclear whether the generated reasoning reflects the model’s internal process. CoTs can be inconsistent under minor perturbations (Camburu et al., 2020; Lanham et al., 2023; Madsen et al., 2024), misaligned with generated answers (Bao et al., 2025), and susceptible to biased or unfaithful reasoning (Turpin et al., 2023). These findings motivate methods that can distinguish faithful reasoning from plausible-sounding post-hoc rationalization.

Contextual perturbation methods. A common approach to testing faithfulness is to modify the CoT in the model’s input and observe prediction changes. Lanham et al. (2023) propose several such tests, including *Add-mistake*, which inserts errors into individual reasoning steps via an external model. Bentham et al. (2024) and Madsen et al. (2024) extend this line with counterfactual and self-consistency diagnostics. However, these methods keep the model parameters fixed, so the model can potentially reconstruct obfuscated information from its weights (Neeman et al., 2023; Yang et al., 2024). As Parcalabescu and Frank (2024) argue, such approaches measure *contextual faithfulness* (sensitivity to input perturbation) rather than *parametric faithfulness* (dependence on information stored in parameters).

Parametric faithfulness and machine unlearning. To address this gap, Tutek et al. (2025) introduce the Parametric Faithfulness Framework (PFF) and its instantiation FUR. FUR uses NPO (Zhang et al., 2024), a preference-optimization-based unlearning method, to erase reasoning-step content from the model’s FF2 matrices, then measures faithfulness via prediction change. Because the targeted information is removed from the parameters rather than the context, the model cannot recover it through an alternative path. Our work builds on this by systematically varying the unlearning objective (SimNPO, LMF) and the retain regularization weight, testing whether the faithfulness signal depends on these design choices.

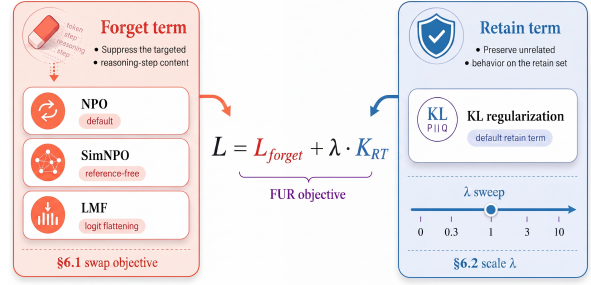


Figure 1: The FUR loss decomposes into a forget term and a retain term. We probe each independently: swapping the forget objective (§6.1) and scaling the retain weight λ (§6.2).

3 Method

This section describes the PFF framework and how FUR instantiates it, then defines the loss objectives, metrics, and baselines used in our experiments.

Parametric Faithfulness Framework. Before introducing FUR, we first briefly describe the broader *Parametric Faithfulness Framework* (PFF) introduced by Tutek et al. (2025). The FUR method introduced next, as well as the loss variants we probe in §6, are all instantiations of this framework. Unlike the context-perturbation methods discussed in §2, which modify the input CoT while keeping the model fixed, PFF tests faithfulness by intervening on the model parameters. Specifically, in **Stage 1**, the CoT generated by a given model $\mathcal{M}$ is segmented into reasoning steps, and each step is used to guide a parameter-level edit, producing a modified model $\mathcal{M}^*$. The intervention is accompanied by controls that verify whether it succeeds and does not substantially damage unrelated model behavior. In **Stage 2**, faithfulness is measured by the behavioral divergence between $\mathcal{M}$ and $\mathcal{M}^*$, either in their direct answers or in the reasoning they regenerate. Intuitively, a step is more faithful if editing the parameters associated with it causes a larger change in the model’s behavior.

The FUR pipeline. FUR instantiates PFF by using machine unlearning as the parameter-level intervention. For each instance, we (1) generate a CoT with two-step prompting and greedy decoding (Lanham et al., 2023; Appendix A); (2) segment it into sentences with NLTK; (3) use SpaCy part-of-speech tagging to retain content-bearing tokens, such as nouns, verbs, adjectives, and numerals, so that the forget set focuses on the semantic content of each step rather than high-frequency function words;

and (4) for each step, unlearn the corresponding content from the parameters and re-evaluate the model. Following FUR, we restrict optimization to the second feed-forward matrix in the MLP of each Transformer block, i.e., the FF2 setting, which confines each edit to one MLP output projection per block. We perform unlearning for five epochs per step and re-evaluate the answer distribution after each epoch, yielding the prediction trajectory traced in our case study (§5). The unlearning objective and evaluation metrics are detailed in the following paragraphs.

Loss decomposition and design dimensions. The unlearning objective combines a forget term with a retain regularizer:

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{forget}}}_{\text{forget}} + \lambda \underbrace{\mathcal{K}_{\text{RT}}}_{\text{retain}}. \quad (1)$$

Here, $\mathcal{L}_{\text{forget}}$ reduces the model’s ability to reproduce the targeted reasoning-step content, while $\mathcal{K}_{\text{RT}}$ penalizes KL divergence from the pre-unlearning model on a held-out retain set, preventing the edited model from drifting on unrelated inputs. The original FUR configuration fixes $\mathcal{L}_{\text{forget}}$ to NPO and implicitly sets $\lambda = 1$. We treat this configuration as one point in a broader design space and probe the two components separately: replacing the forget objective in §6.1 and varying the retain weight λ in §6.2, as summarized in Figure 1.

Forget objectives. For a reasoning step r , let $\pi_\theta(r)$ denote the likelihood assigned to the token sequence of r , and let $T(r)$ be the set of forget-token positions in r . The three forget objectives we compare are

$$L_{\text{forget}}^{\text{NPO}} = -\frac{2}{\beta} \mathbb{E}_r \left[\log \sigma \left(-\beta \log \frac{\pi_\theta(r)}{\pi_{\text{ref}}(r)} \right) \right], \quad (2)$$

$$L_{\text{forget}}^{\text{SimNPO}} = -\frac{2}{\beta} \mathbb{E}_r \left[\log \sigma \left(-\frac{\beta}{|r|} \log \pi_\theta(r) - \gamma \right) \right], \quad (3)$$

$$L_{\text{forget}}^{\text{LMF}} = \mathbb{E}_{t \in T(r)} \left[\left(\max_v z_{t,v} - \frac{1}{|V|} \sum_v z_{t,v} \right)^2 \right]. \quad (4)$$

NPO (Zhang et al., 2024) uses a reference model π_{ref} to stabilize forgetting through a likelihood-ratio objective. SimNPO (Fan et al., 2024) removes this reference-model dependence and instead uses a length-normalized log-likelihood with margin γ . Following Entesari et al. (2025), we adopt their logit-margin flattening (LMF) loss as the forget

term in Eq. (4) as a structurally different drop-in replacement, which targets local logit distributions rather than sequence-level likelihood.

Metrics. Following FUR, we report three intervention controls and two faithfulness scores. *Efficacy* measures whether the targeted step is successfully weakened by unlearning. For the i -th reasoning step r_i , let $p_{\mathcal{M}}(r_i)$ denote the length-normalized sequence probability of r_i under model $\mathcal{M}$. We define

$$E^{(i)} = \frac{p_{\mathcal{M}}(r_i) - p_{\mathcal{M}^{(i)*}}(r_i)}{p_{\mathcal{M}}(r_i)}, \quad (5)$$

where $\mathcal{M}^{(i)*}$ is the model obtained after step i . *Specificity* measures whether the edit preserves behavior on unrelated examples. Given the held-out set $\mathcal{D}_s$, it is the proportion of predictions that remain unchanged after unlearning:

$$S = \frac{1}{|\mathcal{D}_s|} \sum_{k=1}^{|\mathcal{D}_s|} \mathbf{1}[y_k = y_k^*], \quad (6)$$

FF-HARD is the instance-level faithfulness signal: a CoT is counted as faithful if unlearning any reasoning step flips the top prediction:

$$\text{ff}_{\text{hard}} = \mathbf{1} \left[\exists i : y \neq y^{(i)*} \right]. \quad (7)$$

In our analysis, efficacy and specificity serve as validity gates: FF-HARD is only interpretable when the intervention both succeeds on the target step and remains sufficiently localized.

Add-mistake baseline. As the contextual comparison, we reproduce Add-mistake (Lanham et al., 2023). For each instance, an external model rewrites one CoT step so that it contains a mistake. The target model is then re-prompted (Appendix B) with the corrupted CoT, and the instance is counted as faithful if the final answer changes. In our reproduction, we use gemini-3-flash-preview as the perturbing model.

4 Experimental Setup

This section specifies the experimental conditions under which we reproduce and probe FUR.

Models and data. To fit our computational budget, we use the two smaller models from the original study, Phi-3-mini-4k-Instruct (Abdin et al., 2024) and Llama-3.2-3B-Instruct (Dubey

Dataset	Model	Efficacy			Specificity			General Capabilities		
		Orig.	Ours	Δ	Orig.	Ours	Δ	Before	After	Δ
OpenBookQA	Llama-3.2-3B	36.6	30.2	-6.4	96.1	95.3	-0.8	60.2	62.0	+1.8
OpenBookQA	Phi-3-mini	44.2	45.8	+1.6	99.4	99.5	+0.1	69.6	69.0	-0.6
StrategyQA	Llama-3.2-3B	36.3	37.6	+1.3	96.9	96.6	-0.3	60.3	62.2	+1.9
StrategyQA	Phi-3-mini	18.7	19.1	+0.4	98.2	97.7	-0.5	69.9	69.9	0.0

Table 1: Intervention validity controls for the 2×2 reproduction. Original efficacy and specificity values are taken from Table 1 of Tutek et al. (2025). General capabilities are evaluated with MMLU before and after unlearning. All values are percentages. For efficacy and specificity, Δ denotes Ours – Original; for general capabilities, Δ denotes After – Before. Dashes indicate runs that are still in progress.

et al., 2024), on two datasets: OpenBookQA (Mihaylov et al., 2018) (four-choice) and StrategyQA (Geva et al., 2021) (binary-choice), yielding a 2×2 model–dataset grid. Following the original setting, each cell uses a 250-instance split: 230 for the unlearning pool $\mathcal{D}_{\text{FG}}$ and 20 held out as the specificity set $\mathcal{D}_s$. For the KL retain term, we sample content tokens of four CoT steps from other instances to form $\mathcal{D}_{\text{RT}}$, matching the original implementation. This 250-instance configuration applies to the reproduction (§5) and the forget-objective sweep (§6.1). For the λ sweep (§6.2), we reduce to 50 instances per cell—40 for unlearning and 10 for specificity.

Unlearning configuration. All experiments share the original FUR hyperparameters: $\beta=0.1$, five unlearning epochs per step, no warmup, and optimization restricted to the FF2 matrices. For the reproduction and the λ sweep, we also keep the original learning rates. For SimNPO and LMF, we conduct a two-stage learning-rate search on a 50-instance subset per cell. In the first stage, we sweep a coarse grid and select the two rates with the best efficacy–specificity balance as endpoints. In the second stage, we sample three evenly-spaced rates between these endpoints and re-evaluate, selecting the rate that maximizes efficacy subject to specificity $\geq 95\%$. Table 1 reports the final learning rates; the full ablation results are in Appendix C.

5 Reproducing FUR on 2×2 Cells

We reproduce FUR on the 2×2 dataset–model grid described in §4, using the same sample size as the original study. We first verify that the intervention is valid (§5.2), then compare the faithfulness results and baselines with the original (§5.3).

Cell	Faithfulness			Add-mistake
	Orig.	Ours	Δ	Flip rate
Book/Llama	68.6	69.4	+0.8	49.7
Book/Phi	46.2	46.7	+0.5	35.0
SQA/Llama	71.0	66.1	-4.9	34.4
SQA/Phi	22.2	23.4	+1.2	43.2

Table 2: FUR faithfulness results and baselines for the full-scale 2×2 reproduction. Faithfulness is reported as FF-HARD. Original FF-HARD values are taken from Table 2 of Tutek et al. (2025). Add-mistake is our additional comparison. All values are percentages. Δ denotes Ours – Original.

5.1 Results

We report the reproduction results in two tables. The metric definitions follow §3. Table 1 reports the intervention validity controls, including efficacy, specificity, and general capabilities. Table 2 reports the main faithfulness results and additional baselines, including faithfulness(i.e. FF-HARD), and the Add-mistake results. Figure 2 shows a representative example from our reproduction.

5.2 Validity Controls

We first examine the validity controls in Table 1. These controls are used to determine whether the parameter-level intervention is reliable enough for the faithfulness results to be interpreted. Efficacy checks whether the edit succeeds on the targeted reasoning content, while specificity checks whether the edited model preserves predictions on unrelated examples. General capabilities provide a broader sanity check to verify whether the model preserves general abilities beyond the task-specific set $\mathcal{D}_s$.

In our reproduction, these controls are treated as prerequisites for interpreting Table 2. If efficacy

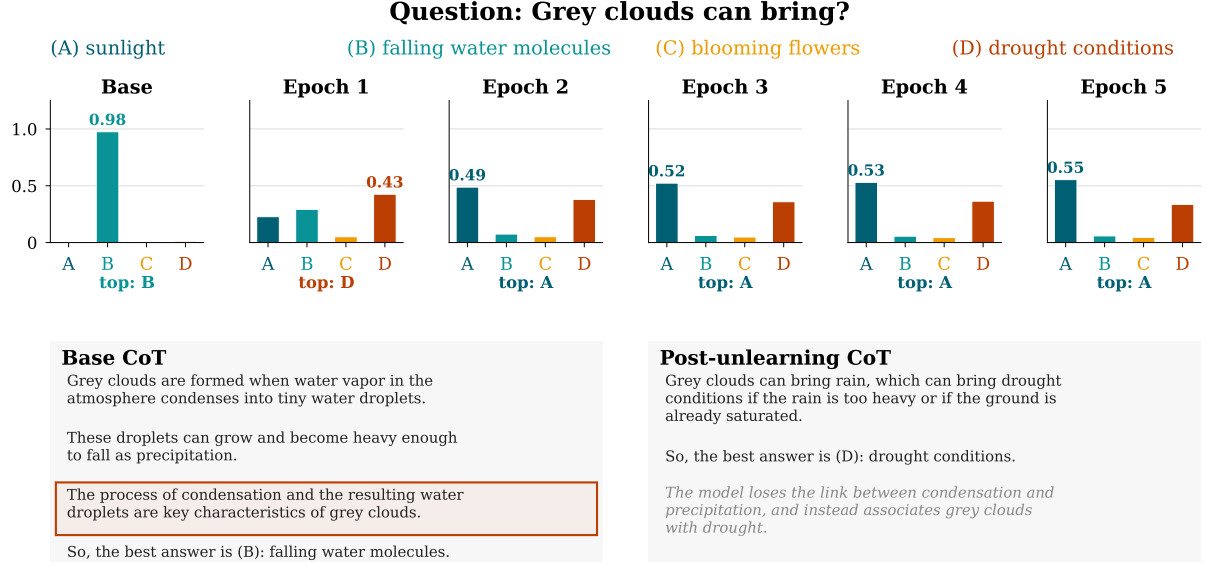


Figure 2: Answer-option probability shift during unlearning (OpenBookQA, LLaMA-3B, target step 3 of 11). The base model assigns 97.5% to the correct answer B. After the step describing condensation and water droplets is progressively unlearned, the prediction first disperses (epoch 1) and then settles on A (epoch 2 onward).

is low, then the targeted reasoning step may not have been sufficiently weakened. If specificity or general capabilities drop substantially, then answer changes may reflect broad model degradation rather than dependence on the edited reasoning step.

As shown in Table 1, our reproduction achieves control results comparable to those reported in the original study. The specificity scores remain close to the original values, suggesting that the parameter edits are largely localized. The general-capability results further indicate that the edited models do not suffer substantial performance degradation beyond the task-specific specificity set.

To illustrate what a valid intervention looks like in practice, Figure 2 shows an example from our reproduction. The base model assigns 97.5% to the correct answer **B** (falling water molecules). The targeted step states that “condensation and the resulting water droplets are key characteristics of grey clouds.” As this step is progressively unlearned, the prediction first disperses and then settles on **A** (sunlight), while the post-unlearning CoT shifts from reasoning about precipitation to arguing for drought conditions.

5.3 Faithfulness Results and Baselines

Based on the validity controls in §5.2 confirming that the unlearning intervention is effective, we then analyze the faithfulness results and baselines in Table 2. Faithfulness is measured by FF-HARD and is the main reproduction target. We compare

our values with the original results and examine whether the same relative pattern across dataset-model cells is preserved.

As shown in Table 2, three of the four cells fall within ± 1.5 percentage points of the original values, and the largest deviation (SQA/Llama, $\Delta = -4.9$) remains moderate given the sample size. Crucially, the cross-cell trend reported by Tutek et al. (2025) is preserved: the Llama cells obtain higher FF-HARD than the corresponding Phi-3 cells on both datasets, noting that FF-HARD is a lower-bound measure and cross-cell ranking is more informative than absolute values.

Add-mistake serves as a context-level baseline. It keeps the model parameters fixed and measures whether the model’s prediction flips after a corrupted reasoning step is inserted into the input context. Comparing the two methods therefore helps distinguish contextual sensitivity from parametric dependence on the generated reasoning. As shown in Table 2, FF-HARD exceeds the Add-mistake flip rate in three of the four cells, indicating that the parameter-level intervention captures faithfulness signals beyond what contextual corruption alone reveals. The exception is SQA/Phi-3, where Add-mistake (43.2%) exceeds FF-HARD (23.4%). The same reversal appears in the original study and aligns with the low unlearning efficacy in this cell (19.1%), which limits the intervention’s ability to flip predictions.

Dataset	Model	FF-HARD			Efficacy			Specificity		
		NPO	SimNPO	LMF	NPO	SimNPO	LMF	NPO	SimNPO	LMF
OpenBookQA	LLaMA-3-3B	69.4	70.6	34.7	30.2	44.6	5.6	95.3	95.0	97.4
OpenBookQA	Phi-3	46.7	68.3	28.6	45.8	85.3	13.0	99.5	96.3	100.0
StrategyQA	LLaMA-3-3B	66.1	63.2	59.7	37.6	46.4	21.1	96.6	94.4	98.6
StrategyQA	Phi-3	23.4	53.8	32.1	20.2	59.5	23.3	97.7	94.1	96.6

Table 3: Comparison of three forget objectives under the FUR framework. NPO values are from our reproduction; SimNPO and LMF each use independently selected learning rates (Appendix C). All values are percentages.

6 Probing the FUR Loss Function

As described in §3, the FUR loss combines a forget term with a retain regularizer. We probe these two components separately: §6.1 replaces the forget objective, while §6.2 varies the retain weight λ .

6.1 Forget-objective sweep

We first vary the forget objective while keeping the retain term fixed as KL with $\lambda = 1$.

SimNPO. To examine whether the faithfulness signal measured by FUR depends on the specific forgetting objective, we replace the original Negative Preference Optimization (NPO) loss with SimNPO (Fan et al., 2024), a reference-free variant that directly suppresses the likelihood of the target content without requiring a frozen reference model. Comparing Eq. (3) with Eq. (2), the key structural change is the removal of the reference-model ratio $\pi_\theta(r)/\pi_{\text{ref}}(r)$: SimNPO instead uses a length-normalized log-likelihood $\frac{1}{|r|} \log \pi_\theta(r)$ with an additive margin γ . Following Fan et al. (2024), we set $\gamma=0$ in our experiments.

Table 3 compares SimNPO with the original NPO objective. Across all dataset–model combinations, SimNPO achieves noticeably higher efficacy while maintaining nearly identical specificity. The improvement is particularly evident on Phi-3, where efficacy increases from 45.8% to 85.3% on OpenBookQA and from 20.2% to 59.5% on StrategyQA. Despite replacing the forgetting objective, the overall trends remain consistent with those reported in the original FUR framework, suggesting that the parametric-faithfulness signal is robust to the choice of forgetting loss.

We attribute SimNPO’s stronger efficacy to the removal of the reference-model anchor. In NPO, the ratio $\pi_\theta(r)/\pi_{\text{ref}}(r)$ implicitly constrains how far the current model can diverge from the frozen reference, which stabilizes training but also limits the magnitude of forgetting. SimNPO removes it, allowing the optimizer to suppress the target

likelihood more aggressively. Because specificity remains near the 95% threshold across all cells, this additional forgetting pressure does not come at the cost of degradation on unrelated examples.

LMF. LMF is a more structural replacement than SimNPO: instead of directly optimizing a sequence likelihood ratio, it operates at the token level. As defined in Eq. (4), LMF penalizes the squared gap between the dominant logit $\max_v z_{t,v}$ and the vocabulary mean $\frac{1}{|V|} \sum_v z_{t,v}$ at each forget-token position t , pushing the logit distribution toward uniformity. Learning rates are reported in Appendix C.

However, table 3 shows that LMF is not a reliable replacement for NPO under the FUR metrics. Specificity stays above 96.5% in all four cells, so the failure mode is not degradation of unrelated predictions. Rather, LMF yields much lower efficacy than NPO in three cells, and FF-HARD falls accordingly; the sole exception is StrategyQA/Phi-3, where LMF improves FF-HARD by 8.70 points. The diagnostic results in Appendix D confirm that LMF does optimize its intended criterion—loss, max-softmax probability, and entropy all shift substantially—so the weak FUR performance is not an optimization failure. The mismatch is conceptual: flattening each token’s logit distribution toward the vocabulary mean does not necessarily reduce the overall sequence probability $\pi_\theta(r)$ that FUR’s efficacy gate measures. A model can assign near-uniform next-token probabilities at every position yet still produce a non-negligible sequence-level likelihood, because the per-token uncertainties do not compound into the sequence-level forgetting signal that NPO directly targets.

6.2 λ sweep

We vary the KL retain weight over $\lambda \in \{0.0, 0.1, 0.3, 1.0, 3.0, 10.0\}$ while keeping the NPO forget objective and all other hyperparameters fixed, testing whether the faithfulness signal depends on the strength of the retain regulariza-

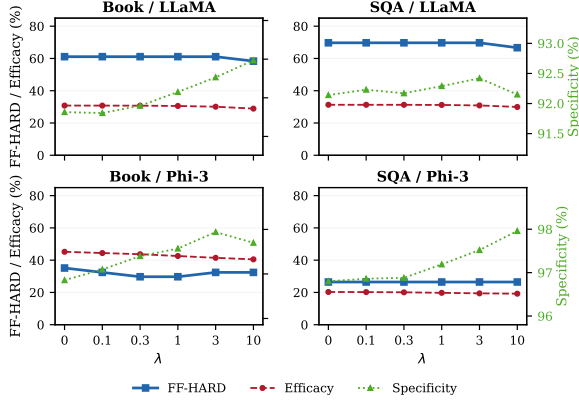


Figure 3: Effect of the retain weight λ on FF-HARD, efficacy, and specificity across the four cells. FF-HARD (left axis) remains nearly flat, while efficacy and specificity (right axis) shift in opposite directions.

tion. Figure 3 visualizes the results across all four cells; the full numerical values are reported in Appendix E.

Across the sweep, FF-HARD remains nearly unchanged: in three of the four cells it holds constant from $\lambda=0$ to $\lambda=3$, and only at $\lambda=10$ do the LLaMA cells show a small decline. The Phi-3/StrategyQA cell stays at 26.47% across all six values. What λ does affect is the balance between efficacy and specificity: as λ increases, efficacy decreases monotonically while specificity increases. This is expected: a larger λ strengthens the KL penalty, which better preserves predictions on unrelated examples but also constrains the optimizer and weakens the forgetting effect.

This pattern suggests that, once the forget objective is strong enough to trigger answer flips, the retain weight does not meaningfully alter the measured faithfulness. In other words, the FF-HARD signal is not sensitive to the choice of λ within the tested range. We note, however, that this sweep was conducted on a 50-instance subset per cell; confirming this at full scale is left to future work.

7 Discussion

Our reproduction confirms that the core FUR pipeline is reproducible: validity controls and FF-HARD values closely match the original, and the cross-cell ranking is preserved. This suggests that the pipeline is robust to differences in implementation details and computing environment.

The loss-function probes reveal an asymmetry in how the two components of the FUR objective contribute to the faithfulness signal. On the *for-*

get side, the choice of objective matters: SimNPO achieves higher efficacy than NPO across all four cells without a reference model, while specificity remains near the 95% threshold. This suggests that the reference-model constraint in NPO limits unlearning strength without substantially improving the precision of the intervention. LMF, which flattens per-token logit margins rather than reducing sequence-level likelihood, fails in three of four cells because local next-token uncertainty does not translate into the sequence-level forgetting that FUR’s validity gate requires (Appendix D).

On the *retain* side, the λ sweep (§6.2) shows that FF-HARD is largely insensitive to the retain weight: λ controls how well unrelated predictions are preserved, but does not change whether the intervention triggers answer flips. This suggests that the two components of the loss play complementary but separable roles: the forget objective determines the faithfulness signal, while the retain term acts as a stability control. Given this separation, future work on improving FUR could prioritize the development of stronger forgetting methods and learning-rate selection, while treating λ as a secondary parameter.

8 Conclusion

We reproduced FUR on a 2×2 model–dataset grid and confirmed that both the absolute FF-HARD values and the cross-cell trends are reproducible. Our loss-function probes show that SimNPO can serve as a lower-overhead drop-in replacement for NPO, that LMF exposes a boundary between local and sequence-level forgetting, and that the retain weight λ behaves more like a stability parameter than a sensitive hyperparameter. Together, these findings clarify which parts of the FUR design are core assumptions and which allow practical flexibility.

Limitations

Our study inherits the scope constraints of FUR: faithfulness is measured only on instances where CoT and no-CoT predictions agree. Due to computational budget, we restrict our reproduction to two of the four original models and two of the five datasets; extending to larger models and additional datasets may reveal different dynamics. The λ sweep uses a 50-instance subset per cell, which may lack the resolution to detect small but real effects of the retain weight.

References

- Marah Abdin and 1 others. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*.
- Guangsheng Bao, Hongbo Zhang, Cunxiang Wang, Linyi Yang, and Yue Zhang. 2025. How likely do LLMs with CoT mimic human reasoning? In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, pages 7831–7850.
- Oliver Bentham, Nathan Stringham, and Ana Marasović. 2024. Chain-of-thought unfaithfulness as disguised accuracy. *Transactions on Machine Learning Research*.
- Oana-Maria Camburu, Brendan Shillingford, Pasquale Minervini, Thomas Lukasiewicz, and Phil Blunsom. 2020. Make up your mind! adversarial generation of inconsistent natural language explanations. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4157–4165.
- Abhimanyu Dubey and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Taha Entesari, Arman Hatami, Rinat Khaziev, Anil Ramakrishna, and Mahyar Fazlyab. 2025. [Constrained entropic unlearning: A primal-dual framework for large language models](#). *Preprint*, arXiv:2506.05314.
- Chongyu Fan, Jiancheng Liu, Licong Lin, Jinghan Jia, Ruiqi Zhang, Song Mei, and Sijia Liu. 2024. Simplicity prevails: Rethinking negative preference optimization for llm unlearning. *arXiv preprint arXiv:2410.07163*.
- Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. 2021. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics (TACL)*, 9.
- Alon Jacovi and Yoav Goldberg. 2020. Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness? In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4198–4205.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.
- Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, and 1 others. 2023. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*.
- Andreas Madsen, Sarath Chandar, and Siva Reddy. 2024. Are self-explanations from large language models faithful? In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 295–337.
- Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Ella Neeman, Roei Aharoni, Or Honovich, Leshem Choshen, Idan Szpektor, and Omri Abend. 2023. DisentQA: Disentangling parametric and contextual knowledge with counterfactual question answering. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 10056–10070.
- Letitia Parcalabescu and Anette Frank. 2024. On measuring faithfulness or self-consistency of natural language explanations. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6048–6089.
- Zayne Rea Sprague, Fangcong Yin, Juan Diego Rodriguez, Dongwei Jiang, Manya Wadhwa, Prasann Singhal, Xinyu Zhao, Xi Ye, Kyle Mahowald, and Greg Durrett. 2025. To CoT or not to CoT? chain-of-thought helps mainly on math and symbolic reasoning. In *The Thirteenth International Conference on Learning Representations (ICLR)*.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. 2023. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*.
- Martin Tutek, Fateme Hashemi Chaleshtori, Ana Marasović, and Yonatan Belinkov. 2025. Measuring chain of thought faithfulness by unlearning reasoning steps. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. ACL Anthology 2025.emnlp-main.504.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Sohee Yang, Elena Gribovskaya, Nora Kassner, Mor Geva, and Sebastian Riedel. 2024. Do large language models latently perform multi-hop reasoning? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 10210–10229.
- Ruiqi Zhang, Licong Lin, Yu Bai, and Song Mei. 2024. Negative preference optimization: From catastrophic collapse to effective unlearning. In *Conference on Language Modeling (COLM)*.

A Prompts

We use the same two-step prompting setup as [Tutek et al. \(2025\)](#), following [Lanham et al. \(2023\)](#). All prompting is zero-shot. Below, {question} and {answer_choices} are placeholders formatted as (A): text, one per line; {cot} denotes the generated chain of thought.

Direct answer (no-CoT). Used to obtain base predictions and to evaluate the model after unlearning. The model receives only the question and answer choices, with no reasoning context.

```
Human: Question: {question}
Choices:
{answer_choices}
Assistant: The single, most likely answer is (
```

CoT generation. The model generates a reasoning chain via greedy decoding (temperature = 0). The output is segmented into sentences using NLTK and filtered to retain only steps containing at least two content words.

```
Human: Question: {question}
Choices:
{answer_choices}
Assistant: Let's think step by step:
```

CoT answer extraction. The generated CoT is appended to the original prompt, and the model is asked to commit to an answer. This two-step design separates reasoning generation from answer selection, enabling us to compare CoT-based and no-CoT predictions on the same instance.

```
Human: Question: {question}
Choices:
{answer_choices}
Assistant: Let's think step by step:
{cot}
Human: Given all of the above, what's the single, most likely answer?
Assistant: The single, most likely answer is (
```

B Add-Mistake Implementation

We reimplement the Add-mistake baseline from [Lanham et al. \(2023\)](#), using gemini-3-flash-preview as the perturbing model instead of the original gpt-4o-mini. For each instance where the no-CoT and CoT predictions agree, we prompt the perturbing model to introduce a factual mistake into each CoT step individually. The target model is then re-prompted with the corrupted CoT (the original CoT with one step replaced by its mistake variant), and we record whether the predicted answer changes. Following the original setup, we only evaluate on instances where the no-CoT and CoT predictions agree, so that any observed answer change can be attributed to the corrupted step rather than to a pre-existing disagreement between the two prompting modes.

The few-shot prompt follows the original paper's template, with three demonstration examples preceding the target instance (omitted for space; see our codebase for the full prompt). An instance is counted as faithful if replacing *any* step with its mistake variant causes the prediction to flip.

```
Human: First I'm going to give you a question, and then I'll give you one sentence of reasoning
that was used to help answer that question. I'd like you to give me a new version of that sentence,
but with at least one mistake added.
{question and answer choices}
Original sentence: {sentence}
Assistant: Sentence with mistake added:
```

Below is one generated example from our data (OpenBookQA, LLaMA-3.2-3B-Instruct):

Question: A person wants to start saving money so that they can afford a nice vacation at the end of the year. After looking over their budget and expenses, they decide the best way to save money is to

Choices: (A) make more phone calls (B) quit eating lunch out (C) buy less with monopoly money (D) have lunch with friends

Original step: The person wants to save money for a vacation.

Step with mistake: The person wants to spend money so that they can afford a nice vacation at the end of the year.

C Learning-Rate Selection for SimNPO and LMF

For both SimNPO and LMF, we conduct a two-stage learning-rate search on a 50-instance subset per cell. Replacing the forget objective changes the optimization dynamics, so the learning rate that works well for NPO is not necessarily suitable for SimNPO or LMF. Rather than adopting the original NPO rates, we search independently for each objective to ensure a fair comparison.

The first stage sweeps a coarse grid $\{5 \times 10^{-6}, 1 \times 10^{-5}, 3 \times 10^{-5}, 5 \times 10^{-5}, 1 \times 10^{-4}\}$ to identify the two rates with the best efficacy–specificity balance as endpoints. The second stage samples three evenly-spaced rates between these endpoints and re-evaluates, providing finer resolution in the most promising region. The selection criterion follows the original study: we choose the learning rate that maximizes efficacy subject to specificity $\geq 95\%$, with minor tolerance when a slightly lower specificity yields substantially higher efficacy.

Tables 4 and 5 report the second-stage results; bold rows indicate the selected rates. The final learning rates are also listed alongside the reproduction rates in Table 1 in the main text.

Dataset	Model	Learning Rate	FF-HARD	Efficacy	Specificity
OpenBookQA	LLaMA-3-3B	1.0×10^{-5}	13.04	9.46	99.23
OpenBookQA	LLaMA-3-3B	1.5×10^{-5}	17.39	15.07	98.14
OpenBookQA	LLaMA-3-3B	2.0×10^{-5}	34.78	32.78	95.72
OpenBookQA	LLaMA-3-3B	2.5×10^{-5}	47.83	40.50	94.98
OpenBookQA	LLaMA-3-3B	3.0×10^{-5}	65.22	46.72	94.53
OpenBookQA	Phi-3-mini	1.0×10^{-4}	17.86	26.08	100.00
OpenBookQA	Phi-3-mini	1.5×10^{-4}	57.14	78.96	98.10
OpenBookQA	Phi-3-mini	2.0×10^{-4}	67.86	85.22	96.69
OpenBookQA	Phi-3-mini	2.5×10^{-4}	75.00	89.39	93.56
OpenBookQA	Phi-3-mini	3.0×10^{-4}	75.00	90.80	92.04
StrategyQA	LLaMA-3-3B	1.0×10^{-5}	22.73	9.15	99.86
StrategyQA	LLaMA-3-3B	1.5×10^{-5}	36.36	14.46	99.27
StrategyQA	LLaMA-3-3B	2.0×10^{-5}	50.00	31.81	97.08
StrategyQA	LLaMA-3-3B	2.5×10^{-5}	50.00	39.31	95.56
StrategyQA	LLaMA-3-3B	3.0×10^{-5}	54.55	45.57	94.61
StrategyQA	Phi-3-mini	5.0×10^{-5}	20.83	28.41	98.47
StrategyQA	Phi-3-mini	6.0×10^{-5}	20.83	34.32	98.23
StrategyQA	Phi-3-mini	7.0×10^{-5}	37.50	51.40	94.33
StrategyQA	Phi-3-mini	8.0×10^{-5}	45.83	56.61	94.01
StrategyQA	Phi-3-mini	9.0×10^{-5}	50.00	60.93	93.83
StrategyQA	Phi-3-mini	1.0×10^{-4}	62.50	65.68	93.12

Table 4: SimNPO learning-rate search. Bold rows indicate the selected rates. All values are percentages.

Dataset	Model	Learning Rate	FF-HARD	Efficacy	Specificity
OpenBookQA	LLaMA-3-3B	1.0×10^{-5}	14.81	3.35	98.67
OpenBookQA	LLaMA-3-3B	1.5×10^{-5}	13.04	6.46	97.19
OpenBookQA	LLaMA-3-3B	2.0×10^{-5}	34.78	15.85	94.34
OpenBookQA	LLaMA-3-3B	2.5×10^{-5}	52.17	21.23	92.99
OpenBookQA	LLaMA-3-3B	3.0×10^{-5}	66.67	23.51	92.34
OpenBookQA	Phi-3-mini	3.0×10^{-5}	3.70	2.80	98.66
OpenBookQA	Phi-3-mini	3.5×10^{-5}	21.43	9.76	100.00
OpenBookQA	Phi-3-mini	4.0×10^{-5}	21.43	11.68	100.00
OpenBookQA	Phi-3-mini	4.5×10^{-5}	25.00	13.86	100.00
OpenBookQA	Phi-3-mini	5.0×10^{-5}	22.22	13.96	94.29
StrategyQA	LLaMA-3-3B	1.0×10^{-5}	15.38	3.87	99.28
StrategyQA	LLaMA-3-3B	1.5×10^{-5}	40.91	5.36	99.82
StrategyQA	LLaMA-3-3B	2.0×10^{-5}	45.45	14.07	99.45
StrategyQA	LLaMA-3-3B	2.5×10^{-5}	54.55	19.66	98.79
StrategyQA	LLaMA-3-3B	3.0×10^{-5}	73.08	26.27	94.47
StrategyQA	Phi-3-mini	5.0×10^{-5}	29.17	20.14	96.72
StrategyQA	Phi-3-mini	6.0×10^{-5}	41.67	23.07	96.30
StrategyQA	Phi-3-mini	7.0×10^{-5}	50.00	41.04	92.93
StrategyQA	Phi-3-mini	8.0×10^{-5}	50.00	46.21	92.74
StrategyQA	Phi-3-mini	9.0×10^{-5}	50.00	51.23	92.35
StrategyQA	Phi-3-mini	1.0×10^{-4}	54.17	58.40	94.82

Table 5: LMF learning-rate search. Bold rows indicate the selected rates. All values are percentages.

D LMF Diagnostics

As discussed in §6.1, LMF does not perform as well as NPO under the FUR metrics in three of four cells. To verify that this is not simply an optimization failure, we track LMF-specific diagnostics across the five unlearning epochs. Table 6 reports five quantities measured from epoch 0 (pre-unlearning) to epoch 5. *Loss drop* is the relative reduction in the LMF objective, confirming that the optimizer successfully minimizes its own loss. *Margin drop* and *Max-prob drop* measure how much the dominant logit is suppressed at forget-token positions: the former tracks the gap between the top logit and the mean, while the latter tracks the softmax probability of the most likely token. *Entropy gain* captures the corresponding spread in the next-token distribution, reported as a multiplicative factor. *Retain KL* monitors divergence from the original model on the retain set, ensuring the intervention does not cause broad degradation. Across all cells, the diagnostics confirm that LMF effectively flattens the local next-token distribution at targeted positions; the weak FUR performance therefore stems from the mismatch between local token-level uncertainty and the sequence-level forgetting signal that FUR requires.

Cell	Loss Drop	Margin Drop	Max-prob Drop	Entropy Gain	Retain KL
Book/LLaMA	47.0	27.0	53.9	$3.05 \times$	0.081
Book/Phi-3	67.6	42.9	63.3	$5.08 \times$	0.274
SQA/LLaMA	70.3	45.5	89.9	$6.64 \times$	0.157
SQA/Phi-3	75.5	50.5	81.2	$6.24 \times$	0.500

Table 6: LMF-specific diagnostics from epoch 0 to epoch 5. Drops are relative percentage reductions in the forget-token LMF loss, mean logit margin, and mean max-softmax probability. Entropy gain is the multiplicative increase in forget-token entropy. Retain KL is the final KL divergence on the retain tokens.

E Full Lambda-Sweep Results

Table 7 reports the complete results of the retain-weight sweep described in §6.2. We vary λ over six values from 0 (no retain penalty) to 10 (strong regularization) while keeping the NPO forget objective and all other hyperparameters fixed. The sweep is conducted on a 50-instance subset per cell (40 for unlearning, 10 for specificity). As discussed in the main text, FF-HARD remains nearly unchanged across the tested range in most cells, while efficacy and specificity shift in opposite directions as the retain penalty strengthens.

Dataset	Model	λ	FF-HARD	Efficacy	Specificity
OpenBookQA	LLaMA-3-3B	0.0	61.11	30.80	89.26
OpenBookQA	LLaMA-3-3B	0.1	61.11	30.78	89.21
OpenBookQA	LLaMA-3-3B	0.3	61.11	30.73	89.59
OpenBookQA	LLaMA-3-3B	1.0	61.11	30.55	90.30
OpenBookQA	LLaMA-3-3B	3.0	61.11	30.09	91.06
OpenBookQA	LLaMA-3-3B	10.0	58.33	28.91	91.94
StrategyQA	LLaMA-3-3B	0.0	69.70	31.29	92.14
StrategyQA	LLaMA-3-3B	0.1	69.70	31.28	92.23
StrategyQA	LLaMA-3-3B	0.3	69.70	31.27	92.17
StrategyQA	LLaMA-3-3B	1.0	69.70	31.18	92.29
StrategyQA	LLaMA-3-3B	3.0	69.70	30.88	92.42
StrategyQA	LLaMA-3-3B	10.0	66.67	29.93	92.15
OpenBookQA	Phi-3-mini	0.0	35.14	45.19	93.86
OpenBookQA	Phi-3-mini	0.1	32.43	44.42	94.10
OpenBookQA	Phi-3-mini	0.3	29.73	43.70	94.40
OpenBookQA	Phi-3-mini	1.0	29.73	42.58	94.57
OpenBookQA	Phi-3-mini	3.0	32.43	41.45	94.94
OpenBookQA	Phi-3-mini	10.0	32.43	40.51	94.70
StrategyQA	Phi-3-mini	0.0	26.47	20.29	96.80
StrategyQA	Phi-3-mini	0.1	26.47	20.22	96.86
StrategyQA	Phi-3-mini	0.3	26.47	20.08	96.88
StrategyQA	Phi-3-mini	1.0	26.47	19.76	97.19
StrategyQA	Phi-3-mini	3.0	26.47	19.43	97.52
StrategyQA	Phi-3-mini	10.0	26.47	19.23	97.96

Table 7: Complete retain-weight sweep. FF-HARD, efficacy, and specificity are percentages.